

This lecture provides a comprehensive overview of inheritance in Python, covering key concepts, implementation details, and related features. The main topics discussed include:

- Introduction to Inheritance:
 - Inheritance is the concept of a class acquiring properties and methods from another class (parent/ancestor).
 - It promotes code reusability by allowing new classes to be based on existing ones.
- Basic Inheritance Implementation:
 - Creating a parent class (e.g., `Mentor`) with attributes (e.g., `name`) and methods (e.g., `show`).
 - Creating a child class (e.g., `EuronMentor`) that inherits from the parent class using `class EuronMentor(Mentor):`. This automatically grants the child class access to the parent's attributes and methods.
 - Demonstration of how the child class can use the parent's constructor (`__init__`) and methods without redefining them.
- Extending Child Classes:
 - Adding new methods specific to the child class, demonstrating that the child class can have its own unique functionality in addition to the inherited features.
- Method Overriding:
 - Redefining a method in the child class that already exists in the parent class. This allows the child class to provide a specialized implementation of the method.
 - Demonstration of how calling the overridden method on an object of the child class executes the child's version of the method, not the parent's.
- The `super()` Function:
 - Using `super()` to call methods from the parent class within the child class, even when the method has been overridden. This allows the child class to extend the parent's functionality while still leveraging the original implementation.
 - Example of using `super()` to call the parent's constructor (`__init__`) to initialize attributes inherited from the parent class, while also adding new attributes specific to the child class.
- Multi-Level Inheritance:
 - Creating a hierarchy of classes where a child class inherits from a parent class, which in turn inherits from another parent class (grandparent). This creates a chain of inheritance.
 - Demonstration of how the child class (e.g., `DSA`) inherits attributes and methods from all its ancestor classes (e.g., `Course`, `Platform`).
- Multiple Inheritance:
 - Creating a class that inherits from multiple independent classes. This allows the child class to combine features from different sources.
 - Example of a class (`EuronSystem`) inheriting from both `Design` and `Curriculum` classes, gaining access to the methods of both.
- Method Resolution Order (MRO):

- MRO defines the order in which Python searches for a method in a class hierarchy when a method is called.
- Using the `__mro__` attribute to inspect the MRO of a class, showing the order in which parent classes are searched for methods.
- Explanation of how MRO helps resolve conflicts when a class inherits methods with the same name from multiple parent classes.
- Learning Objectives and Takeaways:
 - Understand the concept of inheritance and its benefits for code reusability.
 - Implement basic inheritance, method overriding, and the `super()` function.
 - Apply multi-level and multiple inheritance to create complex class hierarchies.
 - Use MRO to understand method resolution order in inheritance scenarios.
-